

UNIVERSIDAD NACIONAL DEL SANTA

FACULTAD DE INGENIERÍA

ESCUELA PROFESIONAL DE INGENIERÍA

DE SISTEMAS E INFORMÁTICA



INFORME DE LABORATORIO SEMANA 8

Alumno:

Corales Samame Santos Dilver

Curso:

Algoritmos Evolutivos

Docente:

Ing. López Heredia Johan Max Alexander

NUEVO CHIMBOTE – PERÚ

2025

Actividad 1: Análisis Comparativo

1. Ejecuta los tres programas y compara los resultados.

CUADRO COMPARATIVO CON LOS DATOS EXTRAIDOS DE LAS EJECUCIONES DE CADA REPRESENTACIÓN

Representación	Distribución de alumnos			Promedio			Desviación Estandar entre promedios	Mejor Fitnes Final	Generaciones para convergencia
	Examen A	Examen B	Examen C	Examen A	Examen B	Examen C			
Binaria	15	11	13	15.13	16.0	15.23	0.3876	-1000	No hay convergencia se mantiene en -1000
Permutación	13	13	13	15.38	15.46	15.38	0.0363	0.2637	Convergencia rápida en generación 0
Real	13	13	13	15.38	15.46	15.38	0.0363	-1.0911	Mejora de -1.1270 hasta -1.0911 en la generación 30 y luego se estabiliza

Actividad 2: Modificación de Fitness

1. En representacion_binaria.py, modifica la función calcular_fitness para:
 - Penalizar grupos con variación alta de notas
 - Premiar diversidad (mezclar alumnos de diferentes rendimientos)

EJECUCIÓN AL REALIZAR LA MODIFICACIÓN

```
REPRESENTACIÓN BINARIA
Problema: Distribuir 39 alumnos en 3 exámenes (A, B, C) de forma equitativa
Cromosoma: 117 bits (39 alumnos x 3 bits cada uno)
Gen: [0,1,0] significa alumno asignado a examen B

Generación 0: Mejor fitness = -8.0637
Generación 20: Mejor fitness = -4.9586
Generación 40: Mejor fitness = -2.7522
Generación 60: Mejor fitness = -1.6107
Generación 80: Mejor fitness = -1.2054

Distribución final:
Examen A: 13 alumnos, promedio = 15.31
  Alumnos: ['Alumno1', 'Alumno3', 'Alumno5', 'Alumno7', 'Alumno10']... (mostrando primeros 5)
Examen B: 13 alumnos, promedio = 19.08
  Alumnos: ['Alumno11', 'Alumno12', 'Alumno15', 'Alumno17', 'Alumno19']... (mostrando primeros 5)
Examen C: 13 alumnos, promedio = 11.85
  Alumnos: ['Alumno2', 'Alumno4', 'Alumno6', 'Alumno8', 'Alumno9']... (mostrando primeros 5)

Verificación de equilibrio:
Desviación estándar entre promedios: 2.9528
```

2. Compara los resultados con la versión original.

	Distribución de alumnos			Promedio			Desviación Estandar entre promedios	Mejor fitness en cada generación				
	Examen A	Examen B	Examen C	Examen A	Examen B	Examen C		0	20	40	60	80
Original	15	11	13	15.13	16	15.23	0.3876	-1000	-1000	-1000	-1000	-1000
Modificado	13	13	13	15.31	19.08	11.85	2.9528	-8.0637	-4.9586	-2.7522	-1.6107	-1.2054

El código modificado distribuye 13 alumnos por examen, mientras que el original tuvo distribuciones desequilibradas (15, 11, 13). Aunque el código modificado muestra una mayor desviación estándar de 2.9528 frente a 0.3876 en el original, lo que indica diferencias en los promedios de los exámenes, mejora la distribución de los alumnos. En cuanto al fitness, el código modificado muestra una mejora progresiva desde -8.0637 en la generación 0 a -1.2054 en la generación 80, mientras que el código original permanece en -1000, sin avanzar, debido a la penalización por la distribución no equilibrada.

Actividad 3: Nuevo Operador Genético

1. En `representacion_real.py`, implementa un operador de mutación diferente:
2. `def mutacion_gaussiana(cromosoma, sigma=0.1)`
3. Prueba con diferentes valores de sigma

Ejecución con sigma = 0.1

```

REPRESENTACIÓN REAL
Problema: Optimizar distribución de alumnos usando pesos probabilísticos
Cromosoma: 117 valores reales (39 alumnos x 3 pesos normalizados)
Gen: [0.2, 0.5, 0.3] representa probabilidades para exámenes A, B, C

Generación 0: Mejor fitness = -1.1270
Generación 30: Mejor fitness = -1.0911
Generación 60: Mejor fitness = -1.0911
Generación 90: Mejor fitness = -1.0911
Generación 120: Mejor fitness = -1.0911

Distribución optimizada:
Examen A: 13 alumnos
  Promedio: 15.38, Varianza: 9.78
  Rango de notas: [10 - 20]
Examen B: 13 alumnos
  Promedio: 15.46, Varianza: 10.25
  Rango de notas: [9 - 20]
Examen C: 13 alumnos
  Promedio: 15.38, Varianza: 11.62
  Rango de notas: [9 - 19]

Análisis de equilibrio:
Promedios por examen: A=15.38, B=15.46, C=15.38
Desviación estándar entre promedios: 0.0363
Diferencia máxima entre promedios: 0.08

```

Ejecución con sigma = 0.5

```

REPRESENTACIÓN REAL
Problema: Optimizar distribución de alumnos usando pesos probabilísticos
Cromosoma: 117 valores reales (39 alumnos x 3 pesos normalizados)
Gen: [0.2, 0.5, 0.3] representa probabilidades para exámenes A, B, C

Generación 0: Mejor fitness = -1.1500
Generación 30: Mejor fitness = -1.0911
Generación 60: Mejor fitness = -1.0911
Generación 90: Mejor fitness = -1.0911
Generación 120: Mejor fitness = -1.0911

Distribución optimizada:
Examen A: 13 alumnos
  Promedio: 15.38, Varianza: 13.31
  Rango de notas: [9 - 20]
Examen B: 13 alumnos
  Promedio: 15.46, Varianza: 9.94
  Rango de notas: [10 - 20]
Examen C: 13 alumnos
  Promedio: 15.38, Varianza: 8.39
  Rango de notas: [9 - 20]

Análisis de equilibrio:
Promedios por examen: A=15.38, B=15.46, C=15.38
Desviación estándar entre promedios: 0.0363
Diferencia máxima entre promedios: 0.08

```

Ejecución con sigma = 0.04

```

REPRESENTACIÓN REAL
Problema: Optimizar distribución de alumnos usando pesos probabilísticos
Cromosoma: 117 valores reales (39 alumnos x 3 pesos normalizados)
Gen: [0.2, 0.5, 0.3] representa probabilidades para exámenes A, B, C

Generación 0: Mejor fitness = -1.0911
Generación 30: Mejor fitness = -1.0911
Generación 60: Mejor fitness = -1.0911
Generación 90: Mejor fitness = -1.0911
Generación 120: Mejor fitness = -1.0911

Distribución optimizada:
Examen A: 13 alumnos
  Promedio: 15.38, Varianza: 10.39
  Rango de notas: [9 - 20]
Examen B: 13 alumnos
  Promedio: 15.38, Varianza: 14.85
  Rango de notas: [9 - 20]
Examen C: 13 alumnos
  Promedio: 15.46, Varianza: 6.40
  Rango de notas: [11 - 19]

Análisis de equilibrio:
Promedios por examen: A=15.38, B=15.38, C=15.46
Desviación estándar entre promedios: 0.0363
Diferencia máxima entre promedios: 0.08

```

Actividad 4: Restricciones Adicionales

1. Modifica `representacion_permutacional.py` para agregar una restricción:
 - Los alumnos con notas < 11 no pueden estar todos en el mismo examen
2. Ajusta la función de fitness para penalizar soluciones que violan esta restricción

Ejecución del Código al realizar los cambios

```
REPRESENTACIÓN PERMUTACIONAL
Problema: Secuenciar alumnos para asignación ordenada a exámenes
Cromosoma: Permutación de 39 índices de alumnos
Decodificación: Posiciones [0-12] -> Examen A, [13-25] -> Examen B, [26-38] -> Examen C

Generación 0: Mejor fitness = 0.1081
Generación 10: Mejor fitness = 0.2637
Generación 20: Mejor fitness = 0.2637
Generación 30: Mejor fitness = 0.2637
Generación 40: Mejor fitness = 0.2637

Asignación final por orden de secuencia:

Examen A: 13 alumnos, promedio = 15.46
Secuencia de alumnos:
  Posición 1: Alumno38 (Nota: 9.0)
  Posición 2: Alumno5 (Nota: 15.0)
  Posición 3: Alumno14 (Nota: 17.0)
  Posición 4: Alumno16 (Nota: 16.0)
  Posición 5: Alumno35 (Nota: 11.0)
  ... (mostrando primeros 5)

Examen B: 13 alumnos, promedio = 15.38
Secuencia de alumnos:
  Posición 1: Alumno1 (Nota: 15.0)
  Posición 2: Alumno8 (Nota: 13.0)
  Posición 3: Alumno12 (Nota: 19.0)
  Posición 4: Alumno9 (Nota: 13.0)
  Posición 5: Alumno28 (Nota: 13.0)
  ... (mostrando primeros 5)
```

Examen C: 13 alumnos, promedio = 15.38

Secuencia de alumnos:

Posición 1: Alumno32 (Nota: 18.0)

Posición 2: Alumno18 (Nota: 16.0)

Posición 3: Alumno34 (Nota: 9.0)

Posición 4: Alumno33 (Nota: 18.0)

Posición 5: Alumno20 (Nota: 19.0)

... (mostrando primeros 5)

Estadísticas finales:

Promedios: A=15.46, B=15.38, C=15.38

Rangos de notas: A=11, B=7, C=11

Desviación estándar entre promedios: 0.0363

Evolución del algoritmo:

Fitness inicial: 0.1081

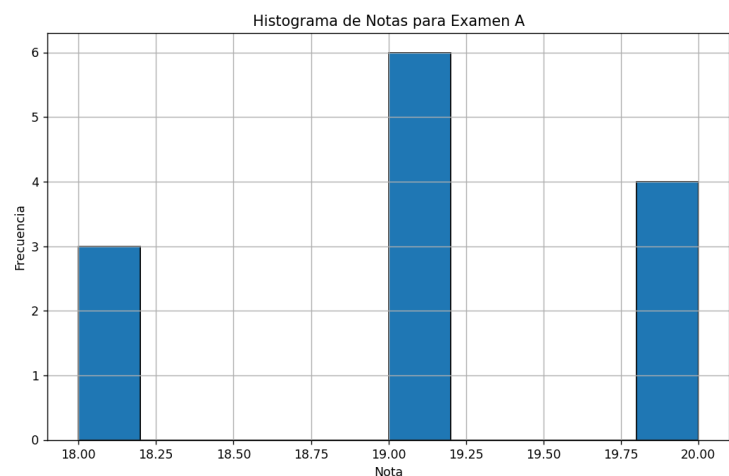
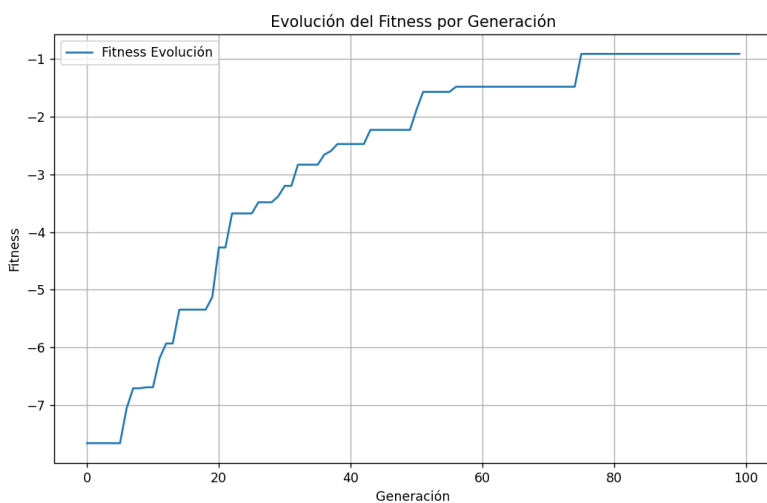
Fitness final: 0.2637

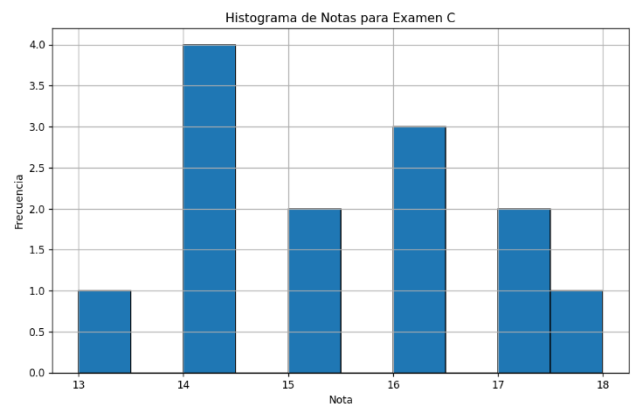
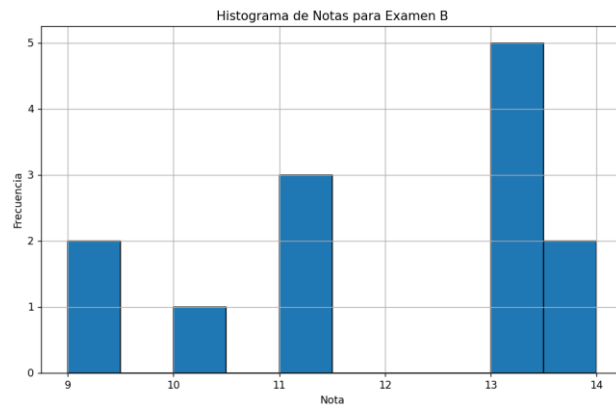
Mejora total: 143.9%

Actividad 5: Visualización (Avanzado)

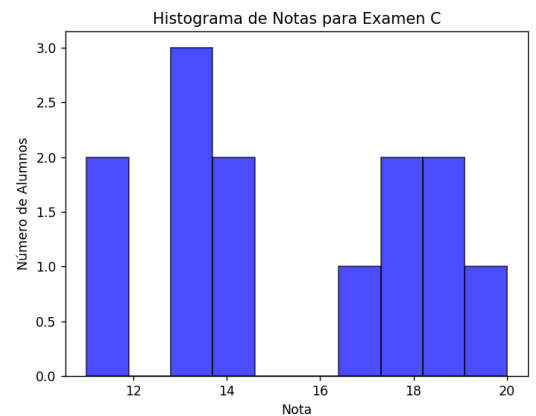
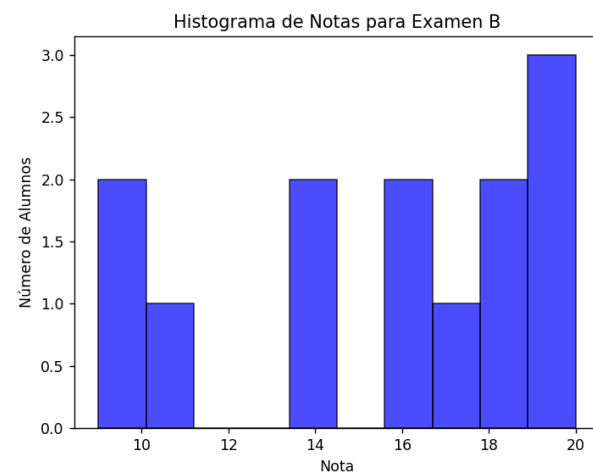
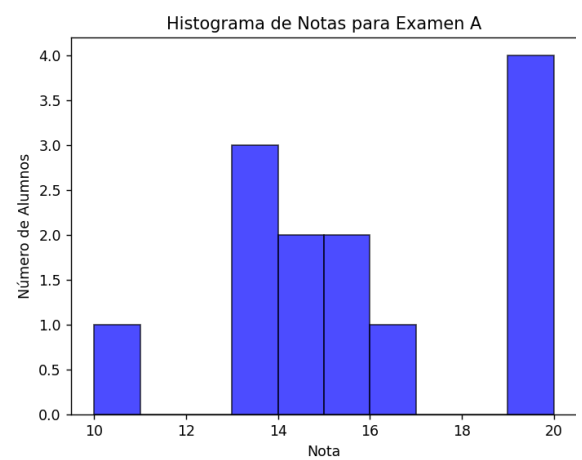
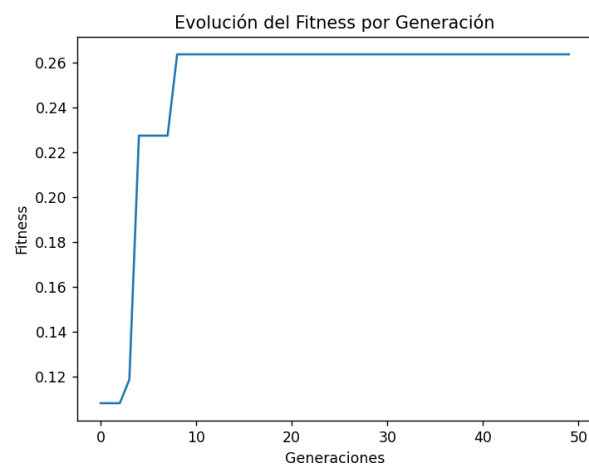
1. Crea un nuevo archivo `visualizacion.py` que:
 - Gráfica la evolución del fitness por generación.
 - Muestre un histograma de notas para examen

REPRESENTACIÓN BINARIA (La grafica son obtenidos a partir de los últimos cambios)

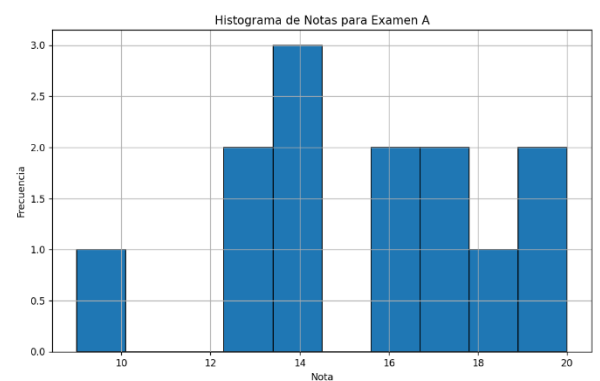
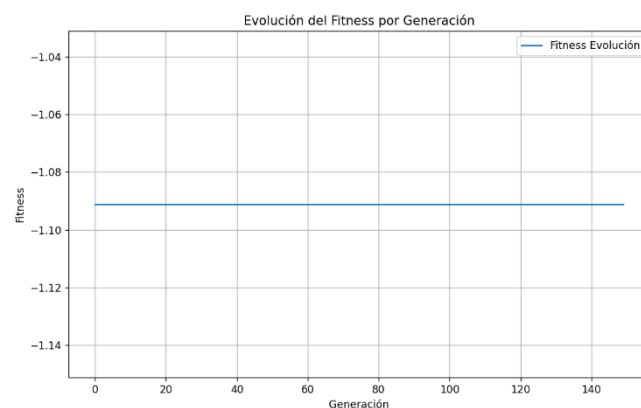


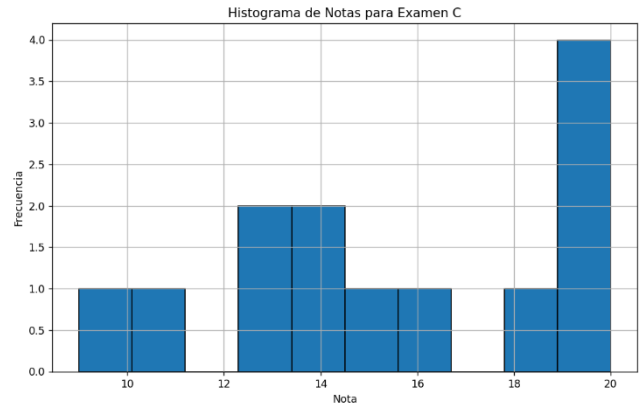


REPRESENTACIÓN PERMUTACIONAL (La grafica son obtenidos a partir de los últimos cambios)



REPRESENTACIÓN REAL (La grafica son obtenidos a partir de los últimos cambios)





Actividad 6: Problema extendido

1. Modifica uno de los programas para distribuir los alumnos en 4 exámenes

REPRESENTACIÓN BINARIA

```

REPRESENTACIÓN BINARIA
Problema: Distribuir 39 alumnos en 4 exámenes (A, B, C, D) de forma equitativa
Cromosoma: 156 bits (39 alumnos x 4 bits cada uno)
Gen: [0,0,1,0] significa alumno asignado a examen C

Generación 0: Mejor fitness = -1000.0000
Generación 20: Mejor fitness = -1000.0000
Generación 40: Mejor fitness = -1000.0000
Generación 60: Mejor fitness = -1000.0000
Generación 80: Mejor fitness = -1000.0000

Distribución final:
Examen A: 12 alumnos, promedio = 16.42
  Alumnos: ['Alumno2', 'Alumno5', 'Alumno8', 'Alumno10', 'Alumno11']... (mostrando primeros 5)
Examen B: 9 alumnos, promedio = 15.22
  Alumnos: ['Alumno14', 'Alumno15', 'Alumno16', 'Alumno18', 'Alumno22']... (mostrando primeros 5)
Examen C: 7 alumnos, promedio = 15.57
  Alumnos: ['Alumno1', 'Alumno3', 'Alumno6', 'Alumno9', 'Alumno17']... (mostrando primeros 5)
Examen D: 11 alumnos, promedio = 14.36
  Alumnos: ['Alumno4', 'Alumno7', 'Alumno21', 'Alumno24', 'Alumno26']... (mostrando primeros 5)
  
```

2. ¿Qué cambios necesitas hacer en el cromosoma?
 - ✓ Aumentar el número de exámenes disponibles de 3 (A, B, C) a 4 (A, B, C, D)
 - ✓ El cromosoma debe cambiar a 4 elementos en vez de 3 para reflejar la inclusión de un cuarto examen
 - ✓ En la decodificación, el índice para cada alumno se debe cambiar para multiplicar por 4 (en lugar de 3) para acomodar la nueva estructura del cromosoma
3. ¿Cómo afecta esto a la convergencia del algoritmo?

En la función **calcular_fitness**, si los exámenes no tienen exactamente 10 alumnos, se asigna un valor de fitness de -1000, este valor es tan bajo que no permite que el algoritmo evolucione correctamente ya que no hay soluciones aceptables en la población inicial, el algoritmo simplemente no puede mejorar porque la penalización es demasiado alta para los cromosomas que no están equilibrados.